Assignment 3: Eyes on the Road

Machine Learning

Chia-Hsuan Hsu (315511079)

Colab notebook: `https://colab.research.google.com/drive/19xXAplgJy12qeEqRIxEpTqfZfyfXiYiI?usp=sharing`

Summary of Results

Model	Validation acc.	Test acc.	TA baseline (val.)
Part II: NumPy MLP (affine–ReLU–affine, 500 hidden)	55.46%	55.10%	21.66%
Part III: CNN (6 conv layers, BatchNorm, augmentation)	91.92%	91.68%	72.42%

Validation accuracy is measured on the last 5,000 training images (also the Kaggle practice leaderboard: 0.5546 for Part II and 0.9192 for Part III). Each test accuracy was computed exactly once, on the final model. All random seeds are fixed to 0.

1 Part I: Neurons by Hand

(a) Architecture

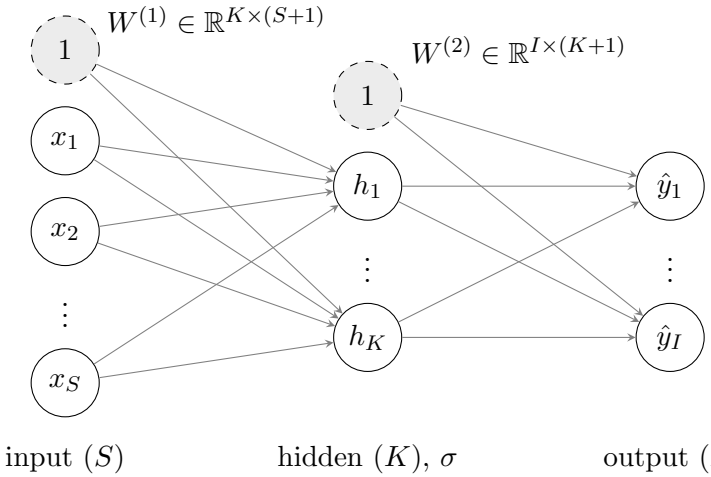


Figure 1: Two-layer network. Dashed nodes are bias units (constant 1); they feed every unit of the next layer but receive no input.

Column 0 of each weight matrix holds the biases, so the parameter count is

$$K(S+1) + I(K+1) = 100 \cdot 3073 + 10 \cdot 101 = 307{,}300 + 1{,}010 = \mathbf{308{,}310}.$$

(b) Backpropagation

Forward pass.

$$z^{(1)} = W^{(1)}x = \begin{bmatrix} 0.1(1) + 0.3(0.5) \\ 0.2(1) + 0.4(0.5) \end{bmatrix} = \begin{bmatrix} 0.2500 \\ 0.4000 \end{bmatrix}, \qquad h = \sigma(z^{(1)}) = \begin{bmatrix} 0.5622 \\ 0.5987 \end{bmatrix},$$

$$\hat{y} = W^{(2)}h + b^{(2)} = 0.5(0.5622) - 0.3(0.5987) = 0.1015, \qquad J = \tfrac{1}{2}(1 - 0.1015)^2 = 0.4037.$$

Backward pass. With $\sigma'(z_i) = h_i(1 - h_i)$: $\sigma'(z_1) = 0.2461$, $\sigma'(z_2) = 0.2403$.

$$\delta^{(2)} = \hat{y} - y = -0.8985, \qquad \delta_1^{(1)} = -0.8985(0.5)(0.2461) = -0.1106, \qquad \delta_2^{(1)} = -0.8985(-0.3)(0.2403) = 0.0648.$$

$$\frac{\partial J}{\partial W^{(2)}} = \delta^{(2)}h^\top = [-0.5051,\ -0.5379], \qquad \frac{\partial J}{\partial b^{(2)}} = -0.8985,$$

$$\frac{\partial J}{\partial W^{(1)}} = \delta^{(1)}x^\top = \begin{bmatrix} -0.1106 & -0.0553 \\ 0.0648 & 0.0324 \end{bmatrix}, \qquad \frac{\partial J}{\partial b^{(1)}} = \begin{bmatrix} -0.1106 \\ 0.0648 \end{bmatrix}.$$

Gradient descent step ($\theta \leftarrow \theta - \lambda \nabla_\theta J$, $\lambda = 0.5$):

$$W^{(1)} = \begin{bmatrix} 0.1553 & 0.3276 \\ 0.1676 & 0.3838 \end{bmatrix}, \quad b^{(1)} = \begin{bmatrix} 0.0553 \\ -0.0324 \end{bmatrix}, \quad W^{(2)} = [0.7526,\ -0.0310], \quad b^{(2)} = 0.4493.$$

Repeating the forward pass gives $z^{(1)} = (0.3744, 0.3271)$, $h = (0.5925, 0.5811)$, $\hat{y} = 0.8772$ and $J = \frac{1}{2}(1 - 0.8772)^2 = \mathbf{0.0075} < 0.4037$, so the loss decreases.

Zero initialization. If all weights are zero, $\delta_i^{(1)} = \delta^{(2)} W_i^{(2)} \sigma'(z_i) = 0$, so $W^{(1)}$ receives no gradient in the first step. More fundamentally, every hidden unit computes the same value ($\sigma(0) = 0.5$) and receives exactly the same gradient, so the units stay identical after every update. This symmetry is never broken and the hidden layer behaves like a single unit. Weights must therefore be initialized with small random values.

(c) Loss Functions

For $\hat{y} = [0.3, 0.2, 0.5]$ and $y = [0, 1, 0]$:

- Predicted class: $\arg\max_k \hat{y}_k$ = class 2 (0-indexed, probability 0.5); the true class is 1, so the prediction is wrong.
- Squared loss: $\sum_k (y_k - \hat{y}_k)^2 = 0.09 + 0.64 + 0.25 = \mathbf{0.98}$ (or 0.49 with the factor $\frac{1}{2}$).
- Cross-entropy loss: $-\sum_k y_k \log \hat{y}_k = -\log 0.2 = \mathbf{1.6094}$.

For $s = [1.2, 1.8, 0.6]$ with true class 1 ($s_y = 1.8$):

$$L = \max(0, 1.2 - 1.8 + 1) + \max(0, 0.6 - 1.8 + 1) = 0.4 + 0 = \mathbf{0.4}, \qquad \frac{\partial L}{\partial s} = [1,\ -1,\ 0].$$

Each class that violates the margin contributes $+1$ to its own score gradient and -1 to the true class; class 2 already satisfies the margin and contributes nothing.

Relation to the SVM. The binary SVM hinge loss is $\max(0, 1 - y\, w^\top x)$ with $y \in \{\pm 1\}$. The multiclass hinge loss generalizes it: the correct score must exceed every other score by a margin of 1. As in the SVM, once the margin is satisfied the loss and its gradient are exactly zero, so only examples near or on the wrong side of the boundary (support vectors) influence training. Cross-entropy, in contrast, never reaches zero and keeps pushing the correct probability towards 1.

(d) Connections

MAP estimation. With $y_n = w^\top x_n + \epsilon_n$, $\epsilon_n \sim \mathcal{N}(0, \sigma^2)$ and prior $w \sim \mathcal{N}(0, \tau^2 I)$,

$$p(w \mid \mathcal{D}) \propto \prod_n \exp\Big(- \frac{(y_n - w^\top x_n)^2}{2\sigma^2} \Big) \exp\Big(- \frac{\|w\|^2}{2\tau^2} \Big),$$

$$- \log p(w \mid \mathcal{D}) = \frac{1}{2\sigma^2} \sum_n (y_n - w^\top x_n)^2 + \frac{1}{2\tau^2} \|w\|^2 + \text{const.}$$

Multiplying by $2\sigma^2$ does not change the minimizer, so

$$w_{\text{MAP}} = \arg\min_w \sum_n (y_n - w^\top x_n)^2 + \lambda \|w\|^2, \qquad \boxed{\lambda = \sigma^2/\tau^2}.$$

More observation noise or a narrower prior gives stronger regularization.

Five linear hidden layers. A composition of affine maps is affine: $W_6(W_5(\cdots(W_1 x + b_1)\cdots) + b_5) + b_6 = W'x + b'$. The network therefore computes $\text{softmax}(W'x + b')$, which is exactly multinomial logistic regression with linear decision boundaries. It is *not* more expressive; if a hidden layer is narrower than the number of classes, W' is even rank-limited and the model is less expressive. Depth only adds expressive power together with non-linear activations.

2 Part II: A Neural Network from Scratch

Data. Images are scaled to $[0, 1]$ and flattened to 3072 values. The first 45,000 training images are used for training and the last 5,000 for validation; the mean training image (computed on the 45,000 images only) is subtracted from all sets. All layers are implemented in `nn_numpy.py` with NumPy only.

(a) Gradient Check

All functions are vectorized. Analytic gradients are compared with centered finite differences ($h = 10^{-5}$) in float64. For a layer with output Z the scalar $\sum Z \odot G$ with a random upstream gradient G is checked. The full networks have two hidden layers (7 and 6 units) with L2 regularization ($\lambda = 0.1$).

Individual functions						
	affine dX	affine dW	affine db	ReLU	sigmoid	
	5.7×10^{-10}	3.9×10^{-10}	5.6×10^{-10}	6.5×10^{-11}	1.4×10^{-9}	
	softmax dS	hinge dS				
	2.4×10^{-9}	7.0×10^{-11}				
Full network	W_1	b_1	W_2	b_2	W_3	b_3
ReLU, softmax	4.3×10^{-9}	1.1×10^{-8}	2.3×10^{-9}	3.4×10^{-10}	2.0×10^{-9}	1.1×10^{-10}
ReLU, hinge	1.7×10^{-9}	2.6×10^{-10}	4.9×10^{-9}	1.2×10^{-10}	1.9×10^{-9}	7.0×10^{-11}
sigmoid, softmax	9.7×10^{-9}	4.3×10^{-9}	2.1×10^{-9}	7.9×10^{-9}	1.6×10^{-9}	9.5×10^{-11}
sigmoid, hinge	1.1×10^{-7}	8.8×10^{-9}	3.4×10^{-8}	1.1×10^{-9}	6.7×10^{-10}	7.0×10^{-11}

Table 1: Relative error $\max_i |a_i - n_i| / \max(10^{-8}, |a_i| + |n_i|)$. The largest value is 1.1×10^{-7}, below the 10^{-6} requirement.

(b) Training

Sanity check. A 3072–100–10 network with no regularization reaches 100% training accuracy (loss 0.0372) on 50 images, confirming that forward and backward passes are correct.

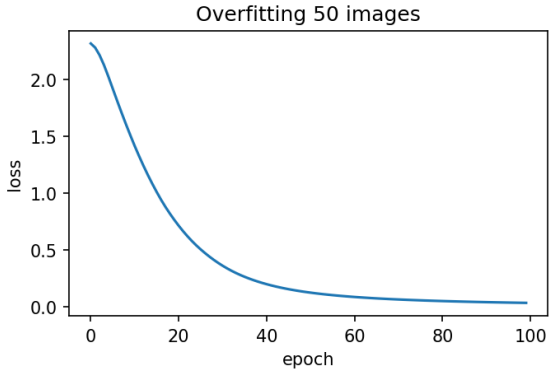


Figure 2: Training loss when overfitting 50 images.

TA baseline. With the prescribed settings (100 hidden units, $\mathcal{N}(0, 10^{-4})$ weights, plain SGD, learning rate 10^{-3}, batch 200, 10 epochs) our run gives 21.66% validation accuracy, identical to the TA baseline. The loss only drops from 2.302 to 2.281: with such a small learning rate and no momentum the network barely moves from its initialization.

Hyperparameter search. All runs use He initialization, SGD with momentum 0.9, batch size 200, learning-rate decay 0.95 per epoch and 20 epochs. Selection uses only the validation set.

Hidden	Learning rate	L2	Train acc.	Final val. acc.	Best val. acc.
100	0.01	10^{-3}	61.00%	52.64%	53.04%
200	0.01	10^{-3}	63.02%	54.38%	54.38%
500	0.01	10^{-3}	67.16%	55.38%	55.38%
500	0.002	10^{-3}	51.84%	48.46%	48.52%
500	**0.05**	$\mathbf{10^{-3}}$	**81.44%**	**55.46%**	**56.08%**
500	0.01	10^{-2}	53.88%	50.40%	50.64%

Table 2: Validation results (training accuracy on a fixed 5,000-image training subset).

A wider hidden layer helps (52.6% → 54.4% → 55.4% for 100, 200, 500 units) because the model has more capacity. A learning rate of 0.002 is too small to converge in 20 epochs (underfitting), while 0.05 gives the best validation accuracy but also the largest train–validation gap (81% vs. 55%), i.e. the start of overfitting. Increasing L2 to 10^{-2} limits the weights too much and lowers both training and validation accuracy. The selected model (500 hidden units, learning rate 0.05, L2 10^{-3}) reaches **55.46%**, far above the 21.66% baseline.

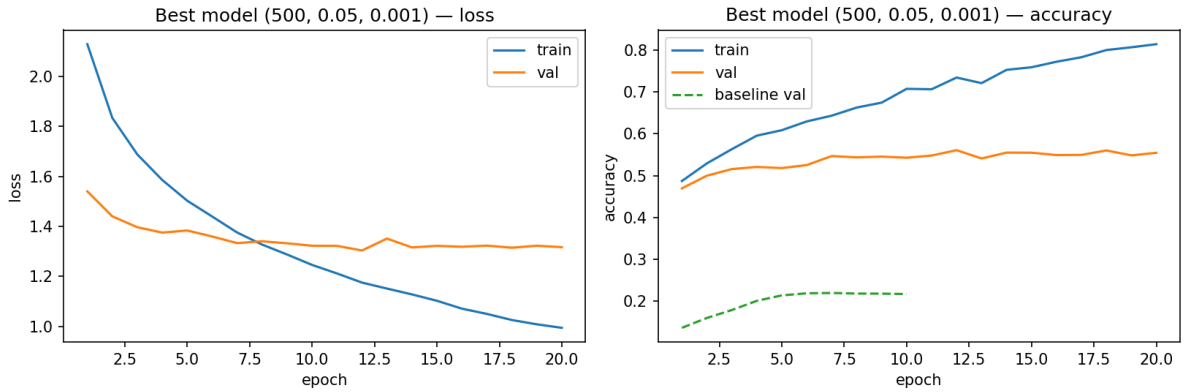


Figure 3: Loss and accuracy curves of the best Part II model; the dashed line is the baseline.

Softmax vs. hinge loss. With the same architecture and hyperparameters, the hinge loss reaches 53.36% validation accuracy versus 55.46% for softmax. The hinge model fits the training set faster (87% vs. 81% training accuracy) but generalizes worse. The hinge loss ignores examples once their margin is satisfied and puts its effort on hard examples, which on this noisy, overlapping data encourages fitting individual training points. The softmax loss gives a smooth probabilistic signal from every example. The two loss values are on different scales and cannot be compared directly.

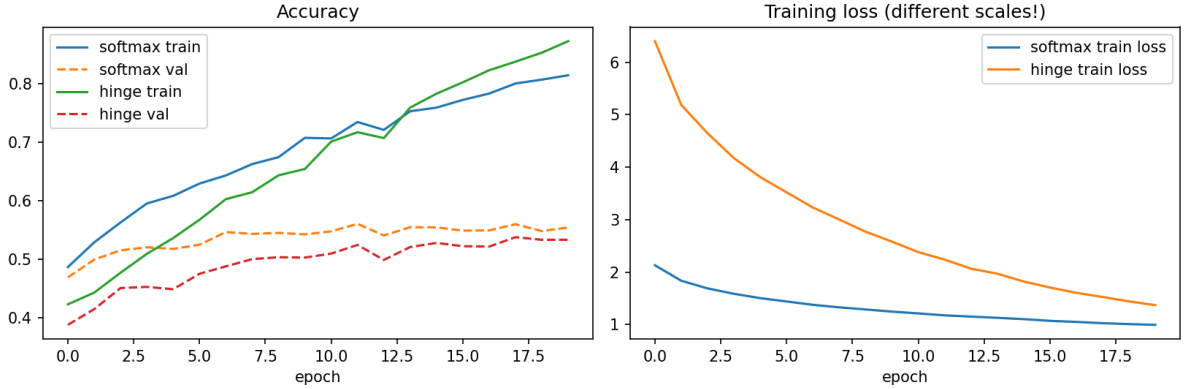


Figure 4: Softmax vs. hinge loss: accuracy (left) and training loss (right).

(c) Design Issues (five hidden layers of 100 units)

The mean absolute gradient of each layer's weights is measured on the first mini-batch before any update. Each network is then trained for 5 epochs (SGD with momentum 0.9, learning rate 10^{-2}, no L2).

Setting	W_1	W_2	W_3	W_4	W_5	W_6	Val. acc. (5 ep.)
ReLU, zero	0	0	0	0	0	0	9.86%
ReLU, small (std 10^{-2})	2.4×10^{-9}	1.2×10^{-8}	1.0×10^{-8}	9.4×10^{-9}	9.8×10^{-9}	3.1×10^{-8}	9.86%
ReLU, large (std 1)	66	695	665	701	736	2015	diverged
ReLU, He	1.4×10^{-3}	1.4×10^{-3}	1.3×10^{-3}	1.3×10^{-3}	1.3×10^{-3}	4.6×10^{-3}	**48.78%**
sigmoid, He	5.6×10^{-6}	9.1×10^{-5}	2.8×10^{-4}	1.0×10^{-3}	3.1×10^{-3}	2.4×10^{-2}	9.50%

Table 3: Mean $|\partial J/\partial W|$ per layer at initialization and validation accuracy after 5 epochs.

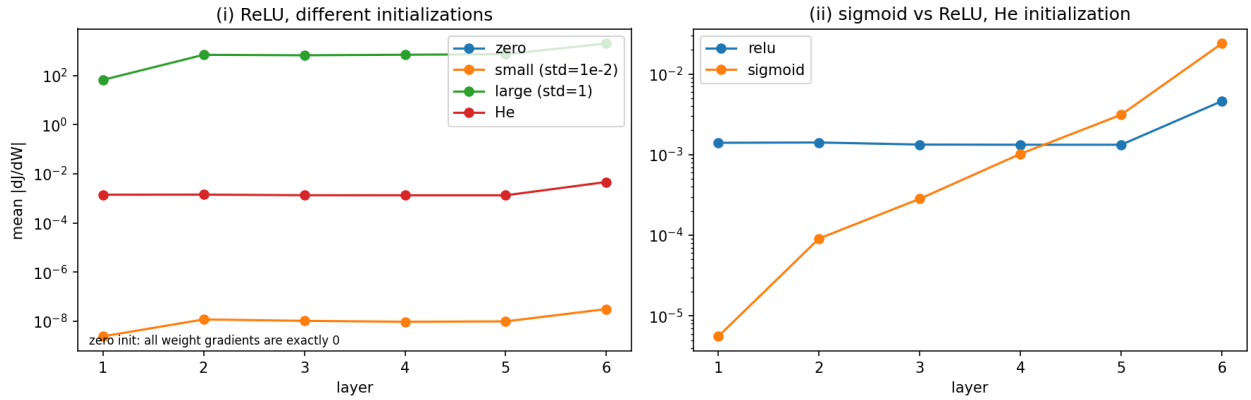


Figure 5: Mean absolute weight gradient per layer. Left: initializations with ReLU (zero init is not shown on the log axis because all gradients are exactly 0). Right: sigmoid vs. ReLU with He initialization.

(i) Initialization.

- *Zero*: every hidden activation is 0, so all weight gradients are exactly 0 and only the output bias learns. The symmetry is never broken; the network stays at chance level (9.86%, loss $\ln 10 \approx 2.303$).
- *Small (std 10^{-2})*: each layer multiplies the signal by roughly $0.01\sqrt{100/2} \approx 0.07$, so activations and back-propagated gradients shrink geometrically. Gradients are around 10^{-8} (*vanishing gradients*) and the network does not learn in 5 epochs.
- *Large (std 1)*: each layer amplifies the signal by about $\sqrt{100/2} \approx 7$ (39 for the 3072-input layer). Gradients reach 10^2–10^3 (*exploding gradients*), the first update overshoots and the loss becomes NaN.
- *He* (std $\sqrt{2/\text{fan_in}}$): the variance of ReLU activations is preserved from layer to layer, so gradients have the same magnitude ($\approx 1.4 \times 10^{-3}$) in every layer and training works (48.78% after 5 epochs).

(ii) Sigmoid vs. ReLU. Under the same He initialization, ReLU gradients are flat across depth, whereas sigmoid gradients shrink by more than three orders of magnitude from the last layer (2.4×10^{-2}) to the first (5.6×10^{-6}). The derivative of the sigmoid is at most 0.25 and is close to 0 when units saturate, so each layer multiplies the back-propagated signal by a small factor. The early layers therefore barely learn and the sigmoid network stays at chance level (9.50%) after 5 epochs. ReLU has derivative 1 for active units and avoids this vanishing-gradient problem.

(iii) L2 regularization. To make overfitting visible, the five-layer ReLU network (He initialization) is trained on a 5,000-image subset for 40 epochs.

L2 strength	Train acc.	Val. acc.	Gap
0	98.80%	37.76%	61.04
10^{-3}	99.20%	39.66%	59.54
10^{-2}	89.40%	**39.76%**	49.64
10^{-1}	10.38%	9.76%	0.62

Table 4: Effect of L2 regularization (5,000 training images, 40 epochs).

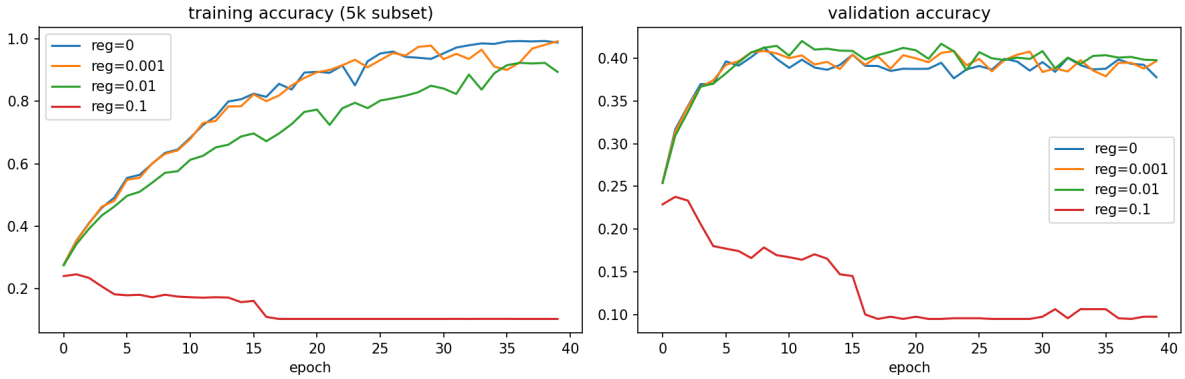


Figure 6: Training (left) and validation (right) accuracy for different L2 strengths.

Without regularization the deep network memorizes the small training set (98.8%) while validation accuracy stays at 37.8%, a clear case of *overfitting*. Moderate regularization (10^{-2}) penalizes large weights, reduces the gap from 61 to 50 points and gives the best validation accuracy. Too strong regularization (10^{-1}) pushes the weights towards zero, the network cannot fit even the training data and both accuracies fall to chance level (*underfitting*). L2 strength therefore trades variance for bias and must be tuned on the validation set.

Test set. The selected Part II model achieves **55.10%** on the test set (evaluated once).

3 Part III: Convolutional Networks

(a) From Pixels to Filters

Output size: $(32 - 5)/1 + 1 = 28$, so six filters give an output of $28 \times 28 \times 6 = 4{,}704$ values. Parameters: $6 \times (5 \cdot 5 \cdot 3 + 1) = \mathbf{456}$. A fully connected layer producing the same 4,704 outputs from the 3,072 inputs needs $3072 \times 4704 + 4704 = \mathbf{14{,}455{,}392}$ parameters, about 31,700 times more.

Why weight sharing helps. Each filter is applied at every position with the same weights, so the number of parameters depends only on the filter size, not the image size. This (i) drastically reduces the number of parameters and the risk of overfitting, (ii) builds in translation equivariance: a feature such as an edge or wheel is detected wherever it appears, so it does not have to be learned separately for every location, and (iii) exploits locality, since nearby pixels are strongly correlated. A fully connected layer ignores the 2-D structure entirely.

(b) My CNN

Layer	Output shape (C, H, W)	Parameters
Input	$(3, 32, 32)$	0
Conv 3×3 (64) – BatchNorm – ReLU	$(64, 32, 32)$	1,856
Conv 3×3 (64) – BatchNorm – ReLU	$(64, 32, 32)$	36,992
MaxPool 2×2 – Dropout	$(64, 16, 16)$	0
Conv 3×3 (128) – BatchNorm – ReLU	$(128, 16, 16)$	73,984
Conv 3×3 (128) – BatchNorm – ReLU	$(128, 16, 16)$	147,712
MaxPool 2×2 – Dropout	$(128, 8, 8)$	0
Conv 3×3 (256) – BatchNorm – ReLU	$(256, 8, 8)$	295,424
Conv 3×3 (256) – BatchNorm – ReLU	$(256, 8, 8)$	590,336
MaxPool 2×2 – Dropout	$(256, 4, 4)$	0
Global average pooling	(256)	0
Linear	(10)	2,570
Total		**1,148,874**

Table 5: Architecture of my CNN. Convolutions use padding 1 and no bias (BatchNorm provides the shift). The TA baseline CNN has 60,362 parameters.

Design choices.

- *Stacked* 3×3 *convolutions*: two 3×3 layers cover a 5×5 receptive field with fewer parameters and an extra non-linearity. Three stages give a receptive field large enough to cover the whole 32×32 image.
- *Doubling channels when halving resolution* (64→128→256) keeps the computation per stage balanced while features become more abstract.
- *Batch normalization* keeps activations well-scaled, reduces sensitivity to initialization and allows a large learning rate.
- *Regularization*: data augmentation (random 32×32 crops from 4-pixel zero-padded images and horizontal flips), weight decay 5×10^{-4} and dropout. The baseline CNN overfits after about 5 epochs (validation loss rises while training loss falls), so regularization is essential for a deeper network.
- *Global average pooling* replaces a large flatten–linear layer and saves parameters.
- *Optimization*: SGD with Nesterov momentum 0.9, a one-cycle learning-rate schedule (max 0.1), batch size 128, 30 epochs.

The network reaches **92.12%** validation accuracy with dropout 0.2, compared with 73.80% for our run of the TA baseline CNN (TA: 72.42%).

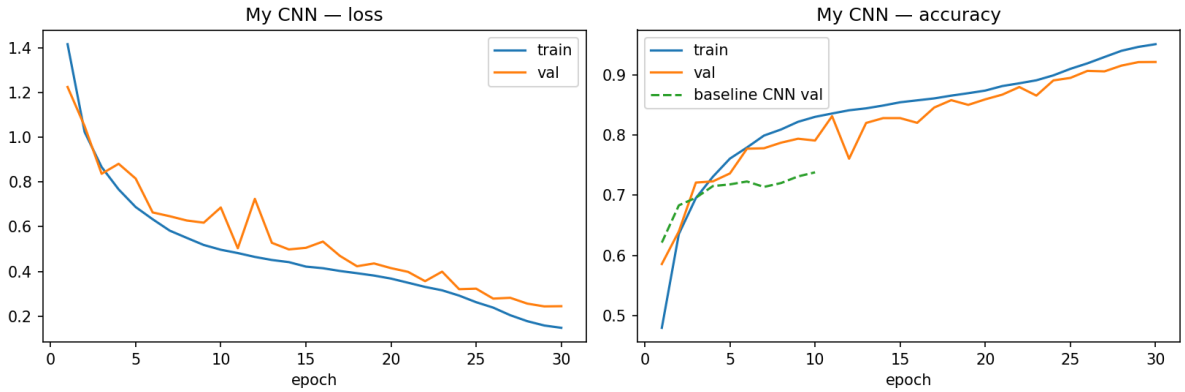


Figure 7: Loss and accuracy of my CNN (dropout 0.2); the dashed line is the baseline CNN.

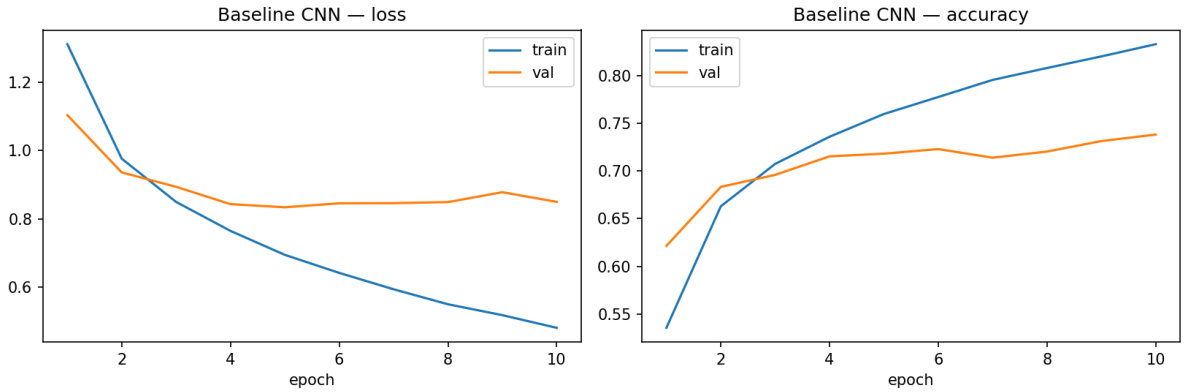


Figure 8: TA baseline CNN: validation loss starts increasing after about 5 epochs (overfitting).

In the first epochs the training accuracy is below the validation accuracy because it is measured on augmented images with dropout active. The one-cycle schedule produces the sharp improvement in the last epochs as the learning rate decays. Unlike the baseline, the validation loss keeps decreasing, so the model does not overfit.

(c) Sensitivity Study: Dropout Probability

All other settings are fixed; each run uses 15 epochs. Training accuracy is measured in evaluation mode (no augmentation or dropout) on 5,000 training images.

Dropout p	Train acc.	Val. acc.	Gap
0.0	96.46%	**90.96%**	5.50
0.1	96.12%	90.68%	5.44
0.2	95.02%	90.30%	4.72
0.3	93.56%	89.32%	4.24
0.5	89.68%	86.58%	3.10

Table 6: Dropout sensitivity (15 epochs).

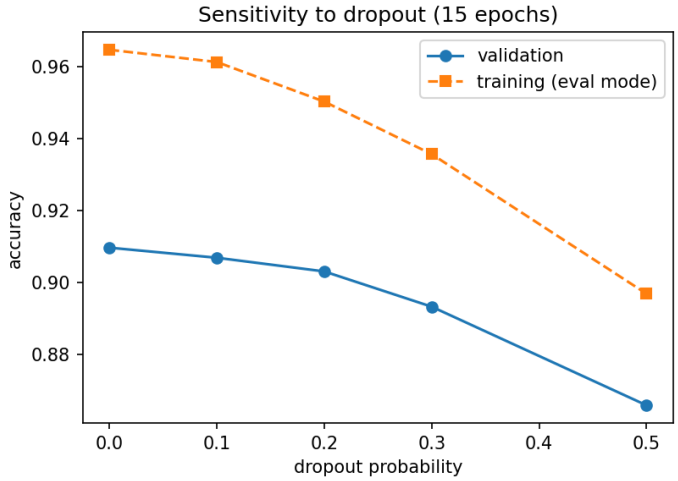


Figure 9: Validation and training accuracy versus dropout probability.

As dropout increases, the train–validation gap shrinks steadily (5.5 → 3.1 points), showing that

dropout reduces overfitting. However, training accuracy falls even faster (96.5% $\rightarrow$ 89.7%), so validation accuracy also drops: with strong dropout the model is *underfitting* rather than overfitting. Because data augmentation, weight decay and batch normalization already regularize the network, and a 15-epoch budget is short, extra dropout mainly removes capacity. The best value on validation is $p = 0$. Note that with the full 30-epoch schedule $p = 0.2$ (92.12%) and $p = 0$ (91.92%) are within the noise of a 5,000-image validation set, so dropout becomes more useful as training gets longer and overfitting becomes more likely.

(d) Final Evaluation

The final model uses the best dropout from (c), $p = 0$, retrained for 30 epochs. Evaluated once on the 10,000-image test set:

Model	Validation acc.	Test acc.	Val. − test
Part II: NumPy MLP	55.46%	55.10%	+0.36
Part III: CNN (final)	91.92%	**91.68%**	+0.24

Comparison with Part II. The CNN improves test accuracy by **36.58 points** (55.10% $\rightarrow$ 91.68%) with fewer parameters than the 500-unit MLP (1.15M vs. 1.54M). The MLP treats each image as an unstructured 3072-dimensional vector and must learn every pattern separately at every location, so it overfits quickly. The CNN exploits locality and weight sharing, builds a hierarchy of features (edges $\rightarrow$ parts $\rightarrow$ objects) and is approximately invariant to small shifts, which matches the structure of images. The final CNN also clearly beats the TA baseline CNN (72.42% on validation).

Was validation accuracy a reliable estimate? Yes. For both models the test accuracy is within 0.4 points of the validation accuracy. The standard error of an accuracy near 92% on 5,000 images is $\sqrt{0.92 \cdot 0.08/5000} \approx 0.4\%$, so the differences are within sampling noise. Validation accuracy is slightly optimistic in both cases, as expected: the configuration was *selected* for its high validation score, which introduces a small positive bias. Since the test set was never used for model selection, the test accuracy is an unbiased estimate of generalization.